Read Only Root Filesystems

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

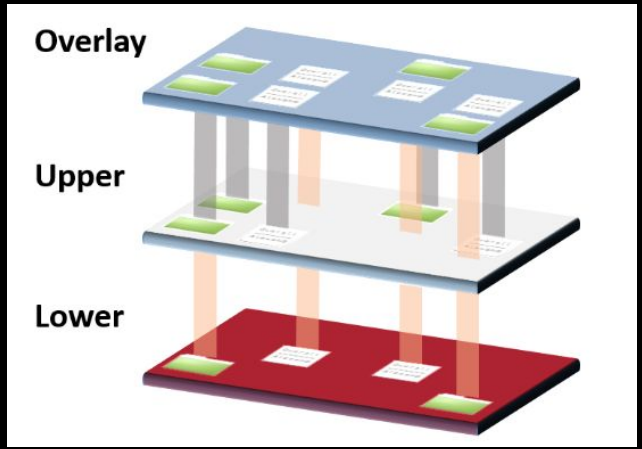
Introduce OverlayFS and tmpfs

Introduce Read Only Root Filesystems

Understand strategies for implementing Read Only Root filesystems in Embedded Devices.

OverlayFS

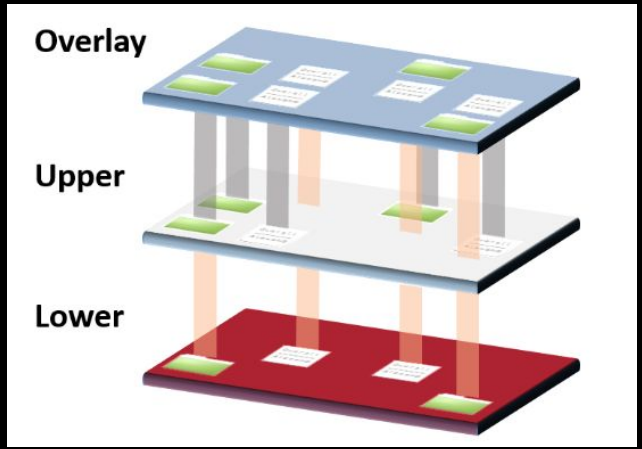
- Mechanism to virtually merge multiple folders, keeping contents separate
- Supported in the Linux kernel after 3.8
- Source directories can be
  - Different volumes
  - different filesystems
- Made up of “Upper” and “Lower”



https://www.datalight.com/blog/2016/01/27/explaining-overlayfs-%E2%80%93-what-it-does-and-how-it-works/

OverlayFS

- Modification to a file in the overlay only in Lower:
  - Copies the content of Lower file to Upper.
  - Modifies the file in Upper
- Modifications to a file in the overlay in Upper:
  - Modifies the file in Upper



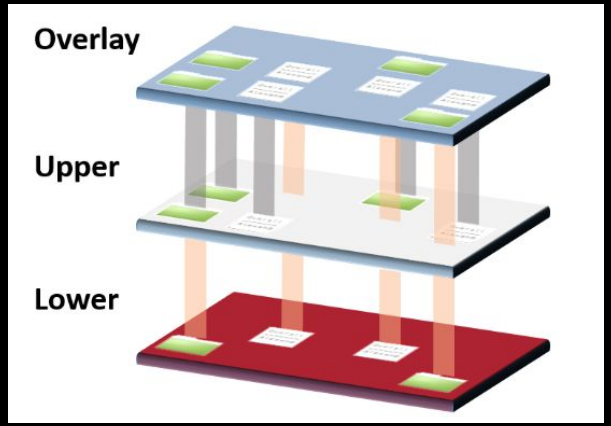
https://www.datalight.com/blog/2016/01/27/explaining-overlayfs-%E2%80%93-what-it-does-and-how-it-works/

OverlayFS

- If we are overlaying two filesystems how can we handle file deletes?
  - Create a “whiteout” file
  - Char device node major/minor 0,0

```
# ls -la /mnt/rw/upper/etc/php.ini
c--------- 1 root root 0, 0 Jul  5 22:29 /mnt/rw/upper/etc/php.ini

# ls -la /mnt/ro/etc/php.ini
-rwxr-xr-x 1 root root 71929 Nov  2 00:39 /mnt/ro/etc/php.ini
# ls /etc/php.ini
ls: cannot access '/etc/php.ini': No such file or directory
```



https://www.datalight.com/blog/2016/01/27/explaining-overlayfs-%E2%80%93-what-it-does-and-how-it-works/
https://www.kernel.org/doc/Documentation/filesystems/overlayfs.txt

OverlayFS

- Also uses a work directory for temporary content
  - Needs to be an empty directory on the same filesystem as upperdir
- Mount an overlay filesystem on /mnt/merged with:
  - mount -t overlay overlay -olowerdir=/path/to/lower,upperdir=/path/to/upper,workdir=/path/to/work /mnt/merged

https://www.kernel.org/doc/Documentation/filesystems/overlayfs.txt

Temporary Filesystems

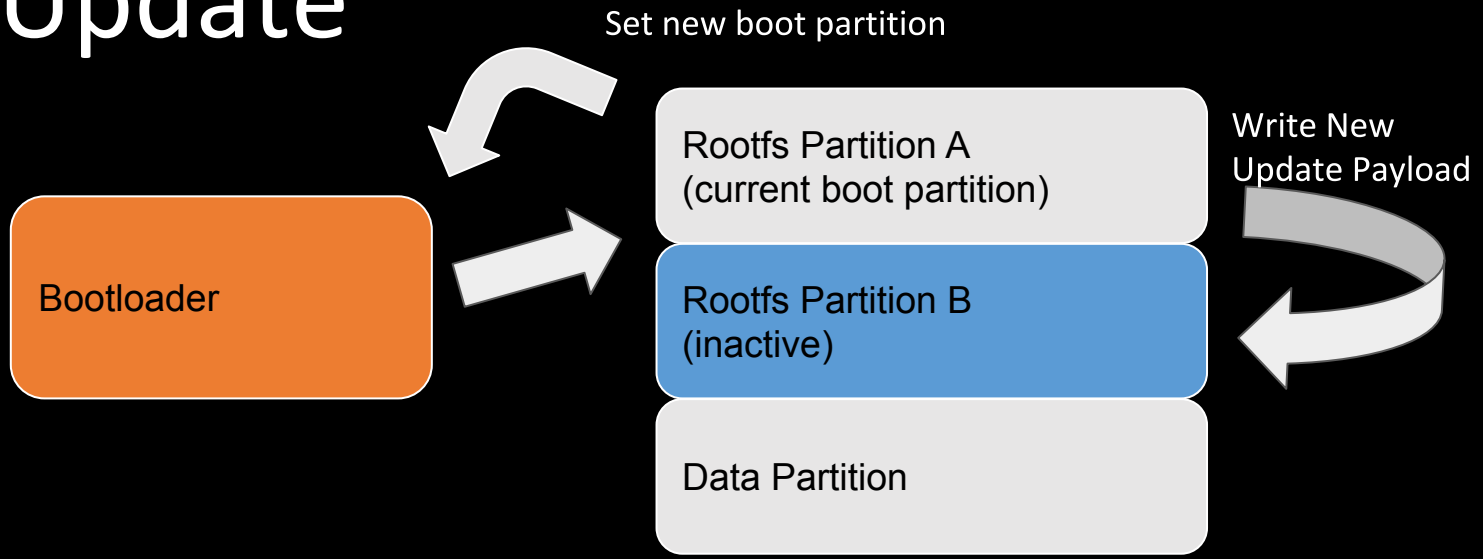
- Some files have a short lifetime, don’t need to be saved across reboots
- tmpfs is useful for this purpose
- Create a filesystem at /mnt/somepath with:
  - mount -o tmpfs tmpfs /mnt/somepath
- Defaults to a max size of half physical RAM, restrict with -o size
  - mount -o tmpfs tmpfs size=1m /mnt/somepath

Read Only Root Filesystems

- Use a read only root filesystem to:
  - Avoid unintentional writes
  - Reduce file system corruption possibilities
    - Power cycles never corrupt root filesystem contents
  - Simplify factory default reset strategies
  - Simplify device update strategies

https://developer.ridgerun.com/wiki/index.php/Read_only_root_file_system

Read Only Root Filesystems for Update



- Rootfs content is completely replaced in this scheme
  - It never makes sense to allow writes to the root filesystem in this scenario

Read only Root Filesystems

- Mount options to the kernel and kernel boot parameters support mounting the root device read only on boot
- We can modify the boot configuration (RPI cmdline.txt file) to include this parameter

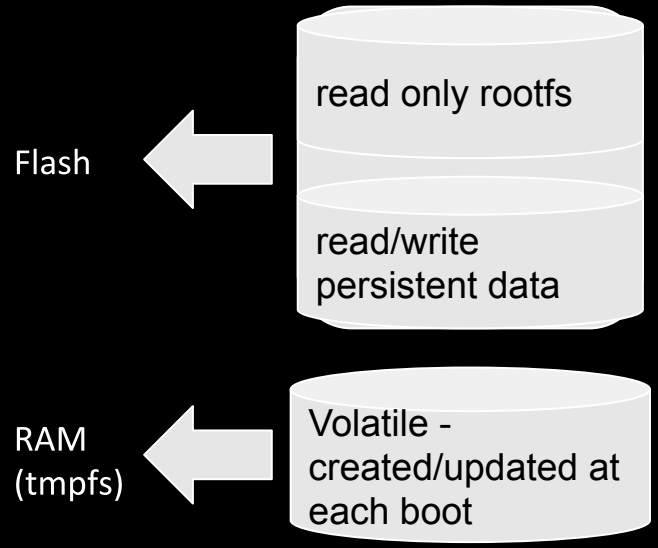
```
ro              [KNL] Mount root device read-only on boot
```

- Is that enough?
  - No, since files will be updated or created at runtime
    - SSH keys for dropbear
    - IP address for dhcp
    - Any software configuration the user can modify on your device.

https://www.kernel.org/doc/Documentation/admin-guide/kernel-parameters.txt
https://www.slideshare.net/chrissimmonds/readonly-rootfs-theory-and-practice
http://hallard.me/raspberry-pi-read-only/
Mastering Embedded Linux Programming Chapter 7 “Making the root filesystem read-only”

Read only Root Filesystems

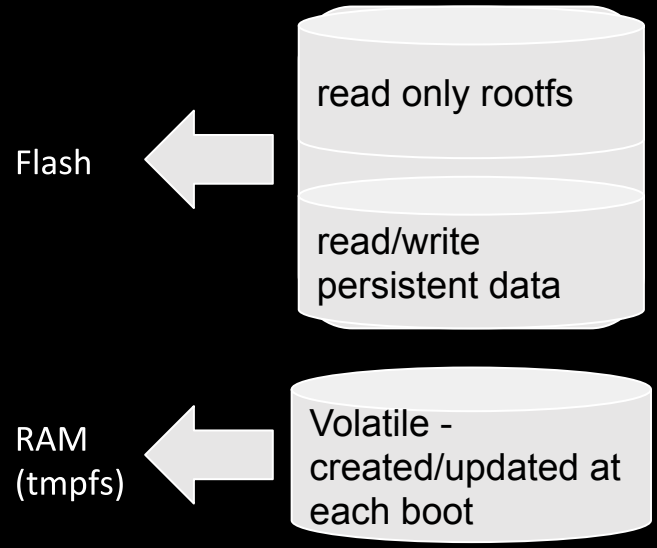
- Split storage needs into categories
- Option 1:
  - Symbolic link specific locations from read only storage to appropriate locations
    - tmpfs for volatile content
    - data partition for persistent content



https://www.slideshare.net/chrissimmonds/readonly-rootfs-theory-and-practice

Read only Root Filesystems

- Split storage needs into categories
- Option 2:
  - Lower layer is located on the read only root partition
  - Place OverlayFS mounts on the root filesystem directories which need writing, place upper layers on:
    - tmpfs for volatile content
    - data partition for persistent content



Read only Root Filesystems

- Yocto supports EXTRA_IMAGE_FEATURES += “read-only-rootfs” to automate some of this setup
- Append or Remove directories from the VOLATILE_BINDS variable to put specific directories in volatile or data partitions

```
VOLATILE_BINDS ?= "\
    /var/volatile/lib /var/lib\n\
    /var/volatile/cache /var/cache\n\
    /var/volatile/spool /var/spool\n\
    /var/volatile/srv /srv\n\
"
```

https://www.yoctoproject.org/docs/latest/dev-manual/dev-manual.html#creating-a-read-only-root-filesystem
https://git.yoctoproject.org/cgit.cgi/poky/plain/meta/recipes-core/volatile-binds/volatile-binds.bb

Chroot

chroot [*OPTION*] *NEWROOT* [*COMMAND* [*ARG*]...]
chroot *OPTION*

- Run command or shell with different root directory
- Any attempts to access / (or any sub-path) after chroot (or in the program specified by COMMAND) accesses NEWROOT instead.
- Useful when you want to simulate a different root for running programs
- Typically requires bind mounts or remounts of temporary filesystems

```
cd /location/of/new/root
mount -t proc /proc proc/
mount --rbind /sys sys/
mount --rbind /dev dev/

mount --rbind /run run/
```

https://linux.die.net/man/1/chroot
https://wiki.archlinux.org/index.php/chroot

Read only Root Filesystems

- Option 3: Overlay the entire root filesystem with another partition
  - Benefit that you don’t need to know in advance which of the directories need to be writable on root.
  - Drawback: Requires a custom init script and chroot sequence on startup (from initramfs):
    - Run from RAM (tmpfs)
    - chroot to RAM
    - Setup overlay rootfs
    - chroot to overlay rootfs
    - Call normal init

http://wiki.psuter.ch/doku.php?id=solve_raspbian_sd_card_corruption_issues_with_read-only_mounted_root_partition